

DOCUMENTATION GESTION DE PROJET

Vite & *Gourmand.*

Méthodologie, outils, traçabilité.

Étudiant · Jassim Elghaouti

Projet · ECF Studi — TP Développeur Web et Web Mobile

Durée indicative · 70 heures

Date · Mai 2026

1. Méthodologie

Sur un projet à un seul développeur de cette taille, j'ai retenu une approche **Kanban simplifié** avec une découpe en blocs fonctionnels. C'est la méthode la plus pragmatique :

- Pas de cérémonies inutiles (un sprint = un développeur n'a pas de sens).
- Visibilité immédiate de ce qui est en cours / fait.
- Possibilité de réordonner les priorités sans rituel.

Le tableau Kanban est tenu dans **Notion**. Les colonnes : *Backlog* → *À faire* → *En cours* → *Revue* → *Fait*.

Espace Notion projet (consultable par le jury)

<https://www.notion.so/3628e177f2a2815ab773e55b796f2602>

Contient : page projet (liens prod + comptes test), database *Tâches* avec **4 vues** (Board Kanban / Calendrier / Timeline Gantt / Table détaillée), 27 tâches réparties sur 7 semaines (1er avril → 20 mai 2026) avec statut, priorité, bloc, période, échéance, heures estimées vs réelles, et tags (sécurité, RGPD, RGAA, UX, critique, doc).

2. Découpage du projet

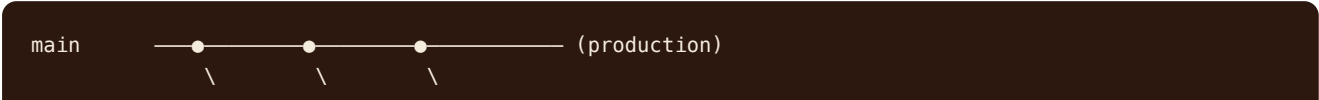
Bloc	Heures estimées	Livrables
1. Cadrage & analyse	4h	User stories, périmètre, stack justifiée
2. Maquettage	8h	Charte graphique, 6 maquettes (3 desktop + 3 mobile)
3. Modélisation BDD	6h	MCD, MLD, MPD, scripts SQL natifs + seed
4. Setup environnement	3h	Repo Git initialisé, gitflow, README, projet Notion
5. Front statique	10h	Toutes les pages HTML/CSS responsive + RGAA
6. Back + Auth	12h	Inscription, login, MDP oublié, rôles, hashage, JWT
7. CRUD menus + filtres dynamiques	8h	Vue globale + détail + filtres AJAX sans reload
8. Workflow commande	8h	Commande complète + mails + statuts

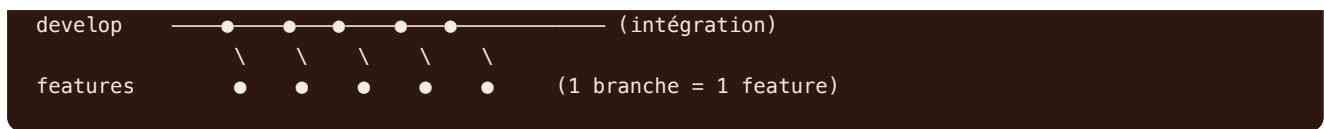
9. Espaces employé + admin	6h	Modération avis, gestion commandes, gestion employés
10. NoSQL + dashboard	3h	MongoDB intégré, graphique commandes/menu, CA
11. Déploiement	2h	App en ligne (Vercel front + back)
12. Documentation + copie ECF	6h	Manuel PDF, doc projet, doc technique, copie remplie
13. Relecture + dépôt	2h	Vérification livrables + soumission Studi
Total	78h	—

3. Outils utilisés

Catégorie	Outil	Usage
Versionning	Git + GitHub	Repo public, gitflow main/develop/feature/*
Hébergement code	GitHub	Repo public obligatoire (cf. énoncé)
Gestion projet	Notion	Kanban + spécifications + journal
Maquettage	Captures Puppeteer + Tailwind config	3 desktop + 3 mobile (rendus fidèles)
IDE	Cursor + Claude Code	Pair-programming IA assisté pour rapidité d'exécution
Communication client	Mail (rôle simulé chef de projet)	—
BDD relationnelle	Supabase (PostgreSQL 17)	Région Paris
BDD NoSQL	MongoDB Atlas (M0 free)	Audit trail + dashboard
Hébergement	Vercel (front + back serverless)	CDN global, deploy git push
Mailer	Resend	Free tier 100 mails/jour

4. Gitflow appliqué





Règles appliquées :

- Une **branche par fonctionnalité** nommée `feature/<nom>`, issue de `develop`.
- Après tests, **merge vers develop** avec message conventionnel (`feat:`, `fix:`, `docs:`, `chore:`, `refactor:`).
- Quand `develop` est stable, **merge vers main** (release).
- Pas de commit direct sur `main`.

Branches créées dans ce projet :

- `main` (release stable)
- `develop` (intégration)
- `feature/database-schema` (schéma BDD + scaffolds + features)

5. Communication & revue

Dans un projet réel d'équipe :

- **Daily standup** de 10 minutes (qui fait quoi, blocages).
- **Pull Requests** avec au moins une review avant merge dans `develop`.
- **Tests d'intégration** automatiques en CI (GitHub Actions).
- **Demos hebdomadaires** avec le chef de projet pour cadrer le delta.

Ici, en projet solo, l'équivalent est :

- Tâche commencée → marquée *En cours* dans Notion.
- Tâche finie → branche mergée + commit conventionnel + colonne Done.
- Tests manuels (parcours visiteur, user, employé, admin) après chaque feature majeure.

6. Risques identifiés & mitigation

Risque	Probabilité	Impact	Mitigation
App pas déployée le jour du jury	Moyenne	Pénalités contractuelles	Déploiement en parallèle dev, tests fréquents en prod
Fuite de secrets dans le repo	Faible	Compromission compte	<code>.env</code> en <code>.gitignore</code> , env vars Vercel chiffrées

Cold start Vercel function trop lent	Moyenne	UX dégradée	Lazy Mongo init, mongodb-memory-server en devDeps (bundle prod léger)
Email non délivré (anti-spam)	Faible	UX dégradée	Domaine Resend par défaut, monitoring console
Pertes des 55 emails Sourcify (projet voisin)	Avérée à mitiger	Donnée client perdue	Backup JSON avant pause du projet Supabase Sourcify

7. Journal de bord (résumé)

```

2026-05-16  setup repo + Supabase + schéma BDD + seed
2026-05-16  scaffold backend Express + frontend Vite (skeletons)
2026-05-16  backend complet : auth, menus, commandes, dashboard, mailer
2026-05-16  frontend complet : 14 pages, 4 rôles, design éditorial gourmand
2026-05-16  refactor backend pour serverless Vercel (mongo lazy)
2026-05-16  déploiement front + back sur Vercel – toutes les routes répondent
2026-05-17  enrichissement Home (parallax hero, atelier collage, KPIs)
2026-05-17  documentation : technique, manuel, charte, gestion projet
2026-05-17  rédaction copie ECF
2026-05-17  soumission Studi
2026-05-20  date limite de rendu (deadline)

```